



Факультет компьютерных наук  
НИУ ВШЭ

24 сентября 2026

# Симуляторы в робототехнике. Знакомство с Webots

Проектный семинар «Мобильные колесные роботы»

Дергачев Степан Алексеевич, [sadergachev@hse.ru](mailto:sadergachev@hse.ru)

Муравьев Кирилл Федорович, [kmuravev@hse.ru](mailto:kmuravev@hse.ru)



# Симуляторы в робототехнике

## Основные плюсы

- Разработка и проверка алгоритмов, если реальные роботы недоступны
- Проверка безопасности алгоритмов без поломки оборудования
- Повторение одного сценария для сравнения алгоритмов, проведение массовых экспериментов
- Сбор данных для алгоритмов, основанных на обучении, массовые запуски при онлайн-обучении
- Доступ к истинному состоянию для проведения экспериментов и точной оценки алгоритмов, регулирование шума входных данных
- Снижение сложности проведения экспериментов в целом, в частности при обучении робототехнике

## Ограничение

Перенос алгоритмов на реального робота требует дополнительной проверки и настройки ([sim2real gap](#)).



## Симуляторы в робототехнике

| Что моделируется? | Примеры                                                                 |
|-------------------|-------------------------------------------------------------------------|
| Среда и сценарий  | Объекты в среде, освещение, начальные условия.                          |
| Кинематика        | Перемещение робота (и его частей) без учета сил.                        |
| Динамика          | Масса, инерция, силы, моменты.                                          |
| Контакты          | Столкновения тел, опора на пол, трение колёс.                           |
| Датчики и приводы | Рендеринг изображения камер, лучи LiDAR, энкодеры, управление моторами. |

Детализацию выбирают под задачу: для поиска пути и для захвата предмета нужны модели разной точности.



## Базовый цикл симуляции

1. Получить доступные управляющие воздействия: моменты/угловые скорости моторов или другие воздействия от контроллеров
2. Рассчитать следующий шаг: применить правила движения, рассчитать силы, контакты и ограничения объектов мира
3. Обновить состояние и время: сохранить новые положения и другие переменные состояния, увеличить время на  $\Delta t$
4. Отрисовать результаты: обновить виртуальные датчики и сформировать изображение.



## Базовый цикл симуляции

### Симуляционное время

- Изменяется дискретными шагами  $\Delta t$ .
- Может идти быстрее или медленнее реального времени.
- На паузе состояние мира не обновляется.

### Точность

- Малый шаг  $\Delta t$  увеличивает объём вычислений.
- Большой шаг может ухудшить устойчивость расчетов физики/контактов.
- Частоты физики, датчиков и отрисовки могут различаться.



## Примеры симуляторов: простые 2D-среды

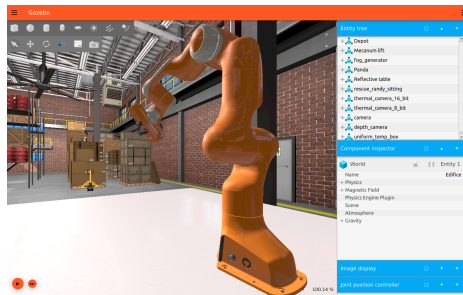
- Не используют сложные физические расчеты/реалистичную графику/сложные модели датчиков
- Удобны для простых и массовых экспериментов
- Редко обладают ROS интерфейсом

| Пример | Основное направление                                                                                          |
|--------|---------------------------------------------------------------------------------------------------------------|
| IR-SIM | Навигация агента/робота на плоскости, различные модели движения роботов, препятствия и датчики                |
| POGEMA | Движение нескольких агентов по клеткам, совместный поиск путей (MAPF), обучение многоагентного RL             |
| CAMAR  | Движение нескольких агентов в непрерывном пространстве, различные модели движения, обучение многоагентного RL |



## Примеры симуляторов: Gazebo

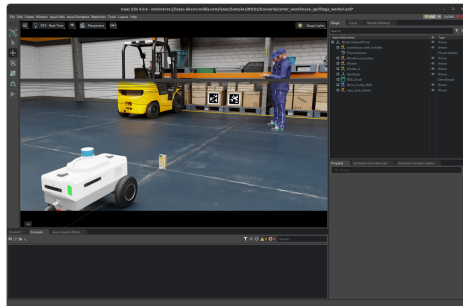
- Стандартный многоцелевой симулятор, входящий в экосистему ROS.
- Моделирование физики, различных датчиков и типов роботов
- Использует форматы файлов, широко распространенные в ROS сообществе.
- Упрощенная визуальная составляющая, для получения более детальной картинки требуется долгая настройка
- Фактически привязан к платформе Linux и ROS (хотя работа без ROS допускается)
- Разделен на два поколения: Classic и Ignition/Sim. Ignition находится в активной разработке, может быть нестабильным, отсутствуют часть функций и неактуальная документация





## Примеры симуляторов: NVIDIA Isaac

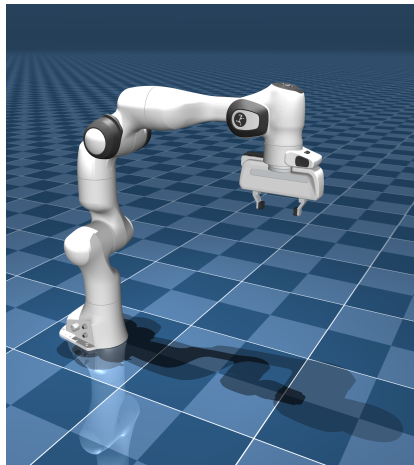
- Многоцелевой симулятор, построен на основе NVIDIA PhysX, обладает реалистичной визуализацией, включая RTX освещение.
- Включает большое количество инструментов для обучения и апробации методов на основе обучения, в том числе RL.
- Есть собственный API и интеграция с ROS. Однако имеет высокий порог входа.
- Требуется производительный компьютер с NVIDIA GPU последних поколений и большим объемом видеопамяти.





## Примеры симуляторов: MuJoCo

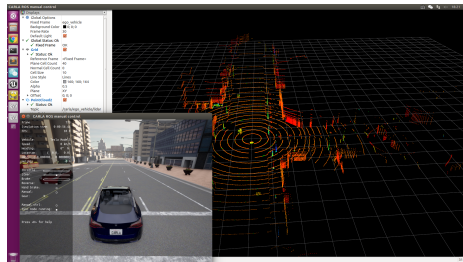
- Симулятор, построенный на физическом движении для точного моделирования сложных систем из твёрдых тел, сочленений и контактов.
- Предоставляет API для управления симуляцией и роботом напрямую, существует интерфейс с ROS2
- Часто применяется для задач оптимального управления и обучения с подкреплением. В частности, для задач манипуляции
- Готовую робототехническую инфраструктуру обычно собирают и проверяют отдельно.





## Примеры симуляторов: CARLA

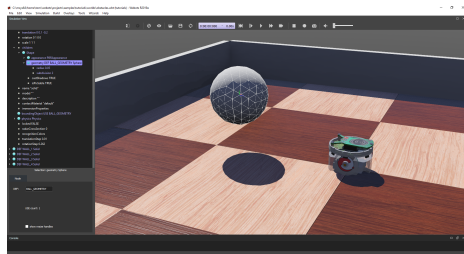
- Специализированная среда для исследований автономного вождения. Содержит моделирование автомобилей (с датчиками) и пешеходов, карты городов и управление транспортными потоками.
- Построена на основе Unreal Engine.
- Предоставляет API для управления симуляцией и автомобилями напрямую, существует интерфейс с ROS2





## Примеры симуляторов: Webots

- Многоцелевой симулятор, включает в себя моделирование физики, различных датчиков и типов роботов, современную и реалистичную визуализацию
- Обладает собственным API и интерфейсом к ROS2, но использует собственные форматы файлов, что немного затрудняет использование с ROS
- Обладает (в среднем) низким порогом входа для обучения и хорошей документацией
- Поддерживает различные платформы: Linux, Windows и macOS.





## Сцена

- Сцена – дерево, состоящая из узлов
- Описывается файлом `.wbt`, используется читаемый формат на основе VRML97
- Включает общие параметры симуляции, настройки точки наблюдения, описание объектов и роботов в сцене

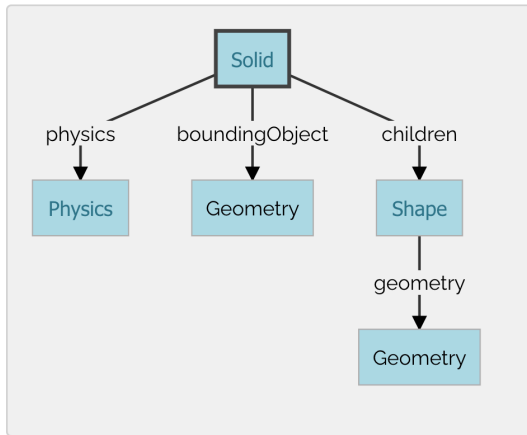
## Узел

- Узел – базовая единица-объект в сцене.
- Узел имеет тип и поля
- Поля задают значения или вложенные узлы.
- Вложенность задаёт структуру и локальные системы координат.
- Шаблон переиспользуемых настраиваемых узлов можно сохранить в файле PROTO.



## Строение узла

- Базовый узел `Solid`
- `physics` – параметры моделирования физики объекта (масса, плотность, материал и т.д.)
- `boundingObject` – модель расчета коллизий и контактов
- `children` – дочерние объекты и/или описание видимой модели узла





## Пример простого объекта

```
1 Solid {
2   translation 1 0 0.2
3   children [
4     Shape {
5       geometry DEF BOX Box {
6         size 0.1 0.1 0.1
7       }
8     }
9   ]
10  boundingObject USE BOX
11  physics Physics {
12    density -1
13    mass 1
14  }
15 }
```

- Куб со стороной 0.1 м и массой 1 кг.
- Механизм `DEF/USE` позволяет повторно использовать геометрию.
- Если убрать `Physics`, то тело не будет двигаться под действием сил
- Без `boundingObject` не будут определяться контакты и столкновения. Поле `physics` тоже должно быть `NULL`.



## Пример PROTO

```
1  PROTO CourseBox [  
2      field SFVec3f translation 0 0 0.2  
3      field SFVec3f size 0.4 0.4 0.4  
4      field SFFloat mass 1  
5  ]  
6  {  
7      Solid {  
8          translation IS translation  
9          # ...  
10     }  
11 }
```

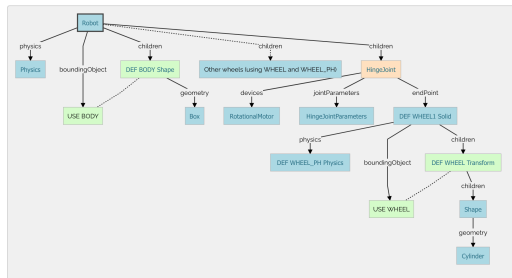
- В PROTO узлу задаются доступные параметры (поля).
- IS связывает значение параметра с полем узла.
- EXTERNPROTO подключает шаблон из файла.
- Экземпляры могут отличаться параметрами.

```
#VRML_SIM R2025a utf8  
EXTERNPROTO "../protos/CourseBox.proto"  
CourseBox { translation 1 0 0.2  mass 2 }
```



## Описание робота

- Робот задается узлом типа `Robot`, который расширяет `Solid`
- У робота есть тело с геометрией и физикой, а также устройства и контроллер.
- `HingeJoint` + `RotationalMotor` + `PositionSensor` моделируют колесо с энкодером
- `Lidar`, `Camera`, `GPS`, `InertialUnit` моделируют датчики.





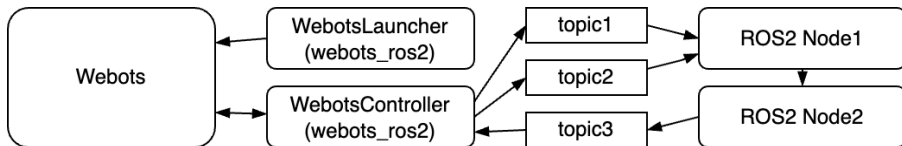
## Контроллер Webots

- Поле `controller` у робота задает управляющую программу, которая может обращаться к датчикам и управлять моторами
- Робот с правами `supervisor TRUE` может читать и изменять через контроллер состояние сцены и всех объектов в ней.
- В режиме `synchronization TRUE` симуляция приостанавливает расчет физического мира и ждет, пока код контроллера выполнит свой текущий шаг и вернет управление.
- Если задать `controller "<extern>"`, то управление будет передано внешней программе с передачей данных по IPC/TCP.



## Связь Webots с ROS2

| Участник         | Роль                                                            |
|------------------|-----------------------------------------------------------------|
| Webots           | Симуляция                                                       |
| WebotsLauncher   | Организует запуск симулятора и открывает нужную сцену           |
| WebotsController | Контроллер робота, который передает данные между ROS2 и Webots  |
| Узлы ROS2        | Читают данные датчиков, выполняют алгоритмы, публикуют команды. |





Спасибо за внимание!